

Persistência de Dados + Mapeamento com JPA

Da memória ao banco: tarefas que sobrevivem ao restart da API

Conceito → bancos relacionais, ORM e JPA • Prática → Spring Data JPA + PostgreSQL + Flyway

A requisição atravessa a infraestrutura

O navegador chega à API pela rede; a API é a única camada que acessa o PostgreSQL.

FLUXO DE UMA REQUISIÇÃO WEB



Regra de segurança: o banco não fica exposto à Internet; só a API conversa com ele.

Portas padrão: memorize o caminho

Porta identifica o serviço na rede. No Docker, a porta do host pode ser diferente da porta interna.

SERVIÇO	PORTA	PAPEL	COMO APARECE AQUI
HTTP	80/TCP	Web sem TLS	navegador → Internet
HTTPS	443/TCP	Web com TLS	produção / domínio
DNS	53/UDP TCP	nome → endereço IP	quando usamos um domínio
Spring Boot	8080/TCP	API local / Tomcat	backend REST da aula
PostgreSQL	5432/TCP	porta padrão do banco	escuta dentro do container
Docker Compose	5433 → 5432	host → container	localhost:5433 → postgres:5432

```
docker-compose.yml: ports: - "5433:5432" → conecte no host em 5433; o PostgreSQL escuta em 5432.
```

Consultar o PostgreSQL no IntelliJ IDEA

O recurso correto é o Database Tools and SQL — normalmente já vem no IntelliJ IDEA Ultimate.

1 Abra os plugins

Ctrl+Alt+S → Plugins

2 Habilite o recurso de banco

Aba Installed → Database Tools and SQL
Clique em Enable; reinicie se for solicitado.

3 Abra a janela Database

View → Tool Windows → Database

Conecte nesta aula

+ → Data Source → PostgreSQL

Host localhost

Port 5433

Database tarefas

User tarefas

Password tarefas

Test Connection

Se pedir, clique em Download driver.

Observação: o Database Tools and SQL não está disponível no IntelliJ IDEA Community Edition.

O que vamos ver hoje

- Parte 1 — Persistência: por que o Map da Aula 07 não basta e como um banco relacional organiza os dados.
- Parte 2 — Mapeamento objeto-relacional: JPA, Hibernate e Spring Data JPA ligando a classe Tarefa à tabela tarefas.
- Parte 3 — Migrations com Flyway e o projeto exemplo_tarefas rodando com PostgreSQL no Docker.
- Vamos sair hoje com o mesmo CRUD de /tarefas, agora gravando de verdade no banco.

1. Persistência de Dados

Bancos relacionais, SQL, PostgreSQL e Docker

2. Mapeamento com JPA

@Entity, @Id, @Column e JpaRepository

3. Migrations e prática

Flyway, ddl-auto e o projeto rodando

De onde viemos

- Na Aula 07, construímos o CRUD de /tarefas em camadas: Controller, Service, Repository e DTO.
- O Repository guardava as tarefas em um Map dentro da memória da aplicação.
- Problema: reiniciou a API, todas as tarefas sumiram. Dois servidores? Cada um com a sua lista.
- Hoje vamos trocar só o Repository: as tarefas passam a morar no PostgreSQL — Controller e contrato da API continuam iguais.

PARTE 1

Persistência de Dados

Guardando informação de forma organizada e permanente, fora da memória da aplicação

Memória x Persistência

- Dados em memória (variáveis, listas, Map) vivem enquanto o processo Java estiver rodando.
- Persistir é gravar os dados em um meio que sobrevive ao fim do processo — disco, via um banco de dados.
- O banco também resolve o que a memória não resolve: acesso concorrente, consultas, integridade e backup.
- Na prática: a API vira 'sem estado' e o banco passa a ser a fonte da verdade.

Map em memória (Aula 07)

Rápido e simples, mas volátil: some a cada restart e não é compartilhado entre instâncias.

Banco de dados (Aula 08)

Persistente, consultável com SQL e compartilhado por todas as instâncias da API.

Banco relacional: tabelas, linhas e colunas

- Um banco relacional organiza os dados em tabelas, parecidas com planilhas bem rígidas.
- Cada coluna tem nome e tipo fixos; cada linha é um registro.
- A chave primária (PRIMARY KEY) identifica cada linha de forma única — é o nosso id.
- Restrições como NOT NULL e DEFAULT garantem integridade no próprio banco, não só no Java.

Tabela

tarefas — o 'molde' dos registros, como uma classe.

Linha (registro)

Uma tarefa específica, como um objeto instanciado.

Coluna

título, concluída, data_prazo... — como os atributos.

Do Java para o SQL: tipos equivalentes

Tipo Java	Tipo PostgreSQL	Onde aparece na Tarefa
Long	BIGINT	id (chave primária gerada pelo banco)
String	VARCHAR(255)	titulo, responsavel
boolean	BOOLEAN	concluida
LocalDate	DATE	dataPrazo → data_prazo, dataCadastro → data_cadastro
BigDecimal	NUMERIC(10,2)	Valores monetários (ex.: preço de um produto)

SQL: o CRUD dentro do banco

- SQL (Structured Query Language) é a linguagem padrão para conversar com bancos relacionais.
- Cada operação HTTP do nosso CRUD vira um comando SQL: POST → INSERT, GET → SELECT, PUT → UPDATE, DELETE → DELETE.
- Hoje não vamos escrever esse SQL à mão no Java — o Hibernate gera por nós. Mas é bom saber ler o que ele gera.

```
INSERT INTO tarefas
  (titulo, responsavel, data_prazo)
VALUES ('Estudar JPA', 'Ana', '2026-10-01');

SELECT id, titulo, concluida FROM tarefas
WHERE responsavel = 'Ana';

UPDATE tarefas SET concluida = TRUE
WHERE id = 1;

DELETE FROM tarefas WHERE id = 1;
```

PostgreSQL + Docker

- PostgreSQL é um banco relacional open source, robusto e muito usado no mercado.
- Ele roda como um servidor separado da API: a aplicação se conecta a ele pela rede (host + porta).
- Em vez de instalar o PostgreSQL na máquina, vamos rodá-lo em um container Docker.
- O Docker Compose descreve o container em um arquivo — o mesmo ambiente para toda a turma.

PostgreSQL

O servidor de banco de dados. Escuta na porta 5432 dentro do container.

Docker Compose

Sobe o banco com um comando: `docker compose up -d`.

Subindo o banco: docker-compose.yml

- image escolhe a versão do PostgreSQL; environment cria banco, usuário e senha na primeira execução.
- ports publica a porta do container no seu computador: 5433 no host → 5432 no container.
- O volume postgres_data guarda os dados fora do container — docker compose down não apaga as tarefas.
- healthcheck avisa quando o banco está pronto para aceitar conexões.

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: tarefas
      POSTGRES_USER: tarefas
      POSTGRES_PASSWORD: tarefas
    ports:
      - "5433:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Memória, tabela ou SQL: o que acontece?

Objetivo: responda cada cenário em uma frase, justificando com o que vimos na Parte 1.

- A API da Aula 07 tinha 10 tarefas cadastradas e foi reiniciada. Quantas tarefas o GET /tarefas devolve? Por quê?
- Escreva o SELECT que lista apenas as tarefas não concluídas da responsável 'Ana'.
- Por que o id é a chave primária da tabela, e não o título?
- Rodamos docker compose down e depois docker compose up -d. As tarefas continuam lá? E com docker compose down -v?

PARTE 2

Mapeamento Objeto-Relacional

Ligando classes Java a tabelas do banco com JPA, Hibernate e Spring Data JPA

O problema: objetos x tabelas

- No Java pensamos em objetos, atributos e referências; no banco, em tabelas, colunas e chaves.
- Os nomes também seguem convenções diferentes: dataPrazo no Java, data_prazo no SQL.
- Converter objeto em linha (e vice-versa) à mão (JDBC puro) é repetitivo e fácil de errar.
- ORM (Object-Relational Mapping) automatiza essa tradução a partir de anotações na classe.

Mundo Java

Classe Tarefa, atributos em camelCase, tipos Long, String, LocalDate.

Mundo SQL

Tabela tarefas, colunas em snake_case, tipos BIGINT, VARCHAR, DATE.

JPA, Hibernate e Spring Data em camadas

- Cada camada se apoia na de baixo — igual à Web em camadas da Aula 07.
- JPA (Jakarta Persistence) é a especificação: define as anotações e as regras.
- Hibernate é a implementação da JPA: lê as anotações e gera o SQL.
- Spring Data JPA fica no topo e cria os repositórios prontos, sem implementação escrita por nós.

Seu código (Service)

Chama `repository.save(tarefa)`

Spring Data JPA

Repositórios prontos: `JpaRepository`

JPA (`jakarta.persistence`)

Especificação: `@Entity`, `@Id`, `@Column`

Hibernate

Implementação: transforma objetos em SQL

JDBC + driver PostgreSQL

Conexão de rede com o banco

Dependências novas no pom.xml

Dependência	Para que serve
spring-boot-starter-data-jpa	Traz JPA, Hibernate e Spring Data JPA já configurados
org.postgresql:postgresql	Driver JDBC: permite ao Java conversar com o PostgreSQL (scope runtime)
flyway-core	Motor de migrations: versiona e aplica os scripts SQL do banco
flyway-database-postgresql	Suporte específico do Flyway ao PostgreSQL

As versões vêm do spring-boot-starter-parent — não é preciso escrever <version> nessas dependências.

Conectando a aplicação ao banco

- `spring.datasource.*` diz onde está o banco: URL JDBC, usuário e senha.
- A URL segue o formato `jdbc:postgresql://host:porta/banco` — aqui, a porta 5433 publicada pelo Docker.
- `ddl-auto=validate`: o Hibernate só confere se as tabelas batem com as entidades; quem cria é o Flyway.
- `open-in-view=false` evita manter a conexão aberta durante toda a requisição.

```
# src/main/resources/application.properties

spring.datasource.url=
    jdbc:postgresql://localhost:5433/tarefas
spring.datasource.username=tarefas
spring.datasource.password=tarefas

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false

spring.flyway.enabled=true
```

Mapeando a entidade Tarefa

- @Entity transforma a classe em uma entidade JPA — cada objeto vira uma linha.
- @Table(name = "tarefas") liga a entidade à tabela criada pela migration.
- @Id + @GeneratedValue(IDENTITY): o banco gera o id no INSERT; o cliente da API nunca envia.
- @Column define nome, obrigatoriedade e se a coluna pode ser alterada depois.

```
@Entity
@Table(name = "tarefas")
public class Tarefa {

    @Id
    @GeneratedValue(
        strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String titulo;

    @Column(name = "data_prazo",
            nullable = false)
    private LocalDate dataPrazo;
    // ... demais atributos
}
```

Anotações de mapeamento mais usadas

Anotação	O que faz	Na classe Tarefa
@Entity	Marca a classe como entidade persistida	Tarefa
@Table	Define o nome da tabela	tarefas
@Id	Marca a chave primária	id
@GeneratedValue	Delega a geração do id (IDENTITY = banco gera)	id
@Column	Configura nome, nullable, updatable, length	data_prazo, data_cadastro
@PrePersist	Método executado antes do INSERT	definirDataCadastro()

Construtor protegido e @PrePersist

- O Hibernate precisa de um construtor sem argumentos para reconstruir objetos lidos do banco.
- Ele pode ser protected: o Hibernate consegue usar, mas o resto do código não cria tarefas 'vazias'.
- @PrePersist roda logo antes do INSERT — lugar certo para preencher valores que a própria entidade decide.

```
// usado pelo Hibernate ao ler do banco
protected Tarefa() {
}

@PrePersist
private void definirDataCadastro() {
    if (dataCadastro == null) {
        dataCadastro = LocalDate.now();
    }
}
```

Spring Data JPA: repositório sem implementação

- O TarefaRepository deixa de ser uma classe com um Map e vira apenas uma interface.
- JpaRepository<Tarefa, Long> = repositório da entidade Tarefa, cujo id é Long.
- Em runtime, o Spring cria a implementação e registra como bean — o Service recebe por injeção de dependência, como na Aula 07.

```
public interface TarefaRepository
    extends JpaRepository<Tarefa, Long> {

    // nada aqui: findAll, findById, save,
    // existsById e deleteById já vêm prontos
}
```

Métodos herdados do JpaRepository

Método	SQL gerado (simplificado)	Usado em TarefaService
findAll()	SELECT * FROM tarefas	listarTodas()
findById(id)	SELECT * FROM tarefas WHERE id = ?	buscarPorId() — devolve Optional
save(nova)	INSERT INTO tarefas (...) VALUES (...)	criar()
save(existente)	UPDATE tarefas SET ... WHERE id = ?	atualizar(), alternarConcluida()
existsById(id)	SELECT count(*) ... WHERE id = ?	remover()
deleteById(id)	DELETE FROM tarefas WHERE id = ?	remover()

Consultas pelo nome do método

- Spring Data lê o nome do método e monta a consulta: `findBy` + atributo + condição.
- Os nomes usam os atributos da entidade (`dataPrazo`), não as colunas do banco (`data_prazo`).
- Palavras como `IgnoreCase`, `False`, `Before`, `And` e `OrderBy...Asc` viram partes do `WHERE` e do `ORDER BY`.

```
public interface TarefaRepository
    extends JpaRepository<Tarefa, Long> {

    List<Tarefa> findByResponsavelIgnoreCase(
        String responsavel);

    List<Tarefa>
        findByConcluidaFalseAndDataPrazoBefore(
            LocalDate data);

    List<Tarefa> findAllOrderByDataPrazoAsc();
}
```

O Service quase não mudou

- O Controller é exatamente o mesmo da Aula 07: mesmas rotas, mesmo contrato.
- No Service, as chamadas ao Map viraram chamadas ao JpaRepository.
- findById devolve Optional: orElseThrow mantém o 404 via TarefaNaoEncontradaException.
- É o ganho de separar camadas: trocar a forma de guardar dados não vaza para o resto da API.

```
public Tarefa criar(TarefaDTO dto) {
    Tarefa tarefa = new Tarefa(null,
        dto.getTitulo(), dto.getResponsavel(),
        dto.getDataPrazo());
    return repository.save(tarefa); // INSERT
}

public Tarefa buscarPorId(Long id) {
    return repository.findById(id) // SELECT
        .orElseThrow(() ->
            new TarefaNaoEncontradaException(id));
}
```

PARTE 3

Migrations e Prática

Versionando o banco com Flyway e colocando o exemplo_tarefas para rodar

Quem cria as tabelas?

- O Hibernate consegue criar/alterar tabelas sozinho (ddl-auto=create ou update) — prático, mas arriscado.
- Com update, ninguém sabe exatamente o que mudou no banco, nem em que ordem; e ele nunca apaga colunas.
- Em projetos reais, o schema é versionado como o código: scripts SQL numerados, no Git — as migrations.
- Neste projeto: Flyway cria e evolui as tabelas; o Hibernate só valida (validate).

ddl-auto = create / update

Hibernate altera o banco sozinho. Bom para protótipo, perigoso em produção.

Flyway + ddl-auto = validate

Mudanças em scripts versionados, revisados e repetíveis em qualquer ambiente.

A primeira migration: V1_criar_tarefas.sql

- Fica em src/main/resources/db/migration — o Flyway procura as migrations ali.
- Cria a tabela tarefas com as mesmas colunas que a entidade Tarefa mapeia.
- Pode também inserir dados iniciais — aqui, duas tarefas de exemplo.

```
CREATE TABLE tarefas (  
    id BIGINT GENERATED BY DEFAULT  
        AS IDENTITY PRIMARY KEY,  
    titulo VARCHAR(255) NOT NULL,  
    concluida BOOLEAN NOT NULL  
        DEFAULT FALSE,  
    responsavel VARCHAR(255) NOT NULL,  
    data_prazo DATE NOT NULL,  
    data_cadastro DATE NOT NULL  
        DEFAULT CURRENT_DATE  
);  
  
INSERT INTO tarefas  
    (titulo, responsavel, data_prazo)  
VALUES ('Preparar slides da Aula 08',  
        'Daniel', CURRENT_DATE + 7);
```

Como o Flyway controla as versões

- Nome do arquivo: V + versão + dois underlines + descrição + .sql.
- No startup, o Flyway compara as migrations da pasta com a tabela flyway_schema_history.
- Só as pendentes são executadas, em ordem de versão — cada uma exatamente uma vez.
- Precisa mudar o banco? Crie a próxima versão (V2, V3...).

```
src/main/resources/db/migration/  
├─ V1__criar_tarefas.sql  
├─ V2__adicionar_prioridade.sql  
└─ V3__...  
  
-- conferindo o histórico  
SELECT version, description, success  
FROM flyway_schema_history;
```

Nunca edite uma migration que já rodou: o Flyway detecta a mudança (checksum) e a aplicação não sobe.

O que acontece quando a API sobe

- 1. O Spring Boot lê o application.properties e abre o pool de conexões com o PostgreSQL.
- 2. O Flyway aplica as migrations pendentes e registra cada uma em flyway_schema_history.
- 3. O Hibernate valida as entidades (@Entity) contra as tabelas — se algo não bate, a aplicação não sobe.
- 4. O Spring Data cria a implementação do TarefaRepository e injeta no TarefaService.
- 5. O Tomcat embutido sobe na porta 8080 e a API começa a atender requisições.

Se o Docker não estiver rodando, a falha acontece já no passo 1: 'Connection refused' na porta 5433.

Rodando o exemplo_tarefas

- Suba o banco primeiro e confira se o serviço aparece como healthy.
- Depois rode a API pelo Maven Wrapper — as migrations rodam sozinhas no startup.
- Abra <http://localhost:8080> para usar a página de cadastro, ou teste pelo Postman.
- Reinicie a API: as tarefas continuam lá. Esse é o objetivo da aula.

```
cd "Aula 08/exemplo_tarefas"

docker compose up -d
docker compose ps

./mvnw spring-boot:run
# Windows: .\mvnw.cmd spring-boot:run

# em outro terminal: consultando o banco
docker compose exec postgres psql -U tarefas \
    -d tarefas -c "SELECT * FROM tarefas;"
```

Do JSON ao INSERT: uma requisição completa

- O contrato HTTP é o mesmo da Aula 07 — o cliente não percebe que agora existe um banco.
- O DTO é validado, o Service monta a entidade e o save() gera o INSERT.
- O id e a data de cadastro voltam preenchidos: vieram do banco.

```
# Requisição
POST /tarefas HTTP/1.1
Content-Type: application/json

{ "titulo": "Estudar JPA",
  "responsavel": "Ana",
  "dataPrazo": "2026-10-01" }

# SQL gerado pelo Hibernate
insert into tarefas (concluida,
  data_cadastro, data_prazo, responsavel,
  titulo) values (?, ?, ?, ?, ?)

# Resposta
HTTP/1.1 201 Created
{ "id": 3, "titulo": "Estudar JPA", ... }
```

Boas práticas de persistência

Prática	Por quê
Versionar o schema com migrations	Todo ambiente chega ao mesmo banco, na mesma ordem, de forma repetível
Usar ddl-auto=validate fora do protótipo	Evita que o Hibernate altere o banco de produção sem revisão
Deixar o banco gerar o id	O cliente nunca decide a chave primária; evita ids duplicados
Regras de integridade também no banco	NOT NULL e DEFAULT protegem os dados mesmo fora da API
Não colocar senha real no application.properties	Em produção, use variáveis de ambiente (veremos em Infraestrutura)

Consultas com métodos derivados

Objetivo: refazer as consultas da Aula 07 usando o banco, com métodos no TarefaRepository em vez de Stream sobre o Map.

- Crie GET /tarefas/buscar?responsavel=Ana usando findByResponsavelIgnoreCase no TarefaRepository.
- Crie GET /tarefas/atrasadas usando findByConcluidaFalseAndDataPrazoBefore(LocalDate.now()) — chamado a partir do Service.
- Faça GET /tarefas devolver as tarefas ordenadas pelo prazo, usando findAllOrderByDataPrazoAsc().
- Para cada endpoint, rode a API e confira no log do Hibernate (spring.jpa.show-sql=true) o SQL gerado.

Nova coluna com uma migration V2

Objetivo: evoluir o banco sem editar a V1 — adicionando a prioridade da tarefa de ponta a ponta.

- Crie V2__adicionar_prioridade.sql adicionando a coluna prioridade (INTEGER, NOT NULL, DEFAULT 3).
- Mapeie o atributo prioridade na entidade Tarefa com @Column.
- Adicione prioridade ao TarefaDTO com @Min(1) e @Max(5) e use no criar/atualizar do Service.
- Reinicie a API e confira a V2 em flyway_schema_history. As tarefas antigas continuam lá?

```
-- V2__adicionar_prioridade.sql
ALTER TABLE tarefas
    ADD COLUMN prioridade INTEGER
    NOT NULL DEFAULT 3;
```

RECAPITULANDO

O que levamos para casa

- Persistir é tirar os dados da memória da aplicação: o banco vira a fonte da verdade e sobrevive ao restart.
- ORM traduz objetos em linhas: JPA define as anotações, Hibernate gera o SQL, Spring Data entrega o repositório pronto.
- O schema do banco é versionado como código, com migrations Flyway; o Hibernate só valida.
- Trocamos só o Repository — Controller e contrato da API continuaram os mesmos. Esse é o valor das camadas.

Dever de casa

Finalize os 3 exercícios (check rápido, consultas derivadas e a migration V2 com prioridade) antes da próxima aula.

Para ir além

- Documentação do Spring Data JPA — repositórios e métodos de consulta derivados.
- Documentação do Flyway — convenções de nomes e comandos de migration.
- Documentação do PostgreSQL — tipos de dados e comandos SQL.
- Repositório da disciplina: github.com/placidodaniel/backend-aulas-satc